JEGYZŐKÖNYV

Webes adatkezelő környezetek

Féléves feladat

Gabonatelep nyilvántartása

Készítette: **Deák Csaba**
Neptunkód: **ILQ2OX**
Dátum: **2026. november.10.**

Miskolc, 2026

Tartalomjegyzék

Bevezetés 3

A feladat leírása 3

1. Az XML adatkezelő rendszer tervezése és megvalósítása 4

1.1 Az adatbázis ER modell tervezése 4

1.2 Az adatbázis konvertálása XDM modellre 5

1.3 Az XDM modell alapján XML dokumentum készítése 8

1.4 Az XML dokumentum alapján XMLSchema készítése 8

2. A DOM program készítése 10

2.1 Adatolvasás 10

2.2 Adat-lekérdezés 12

2.3 Adatmódosítás 14

Bevezetés

A jegyzőkönyv a Webes adatkezelő környezetek tantárgy féléves feladatát dokumentálja. A feladat egy XML-alapú adatkezelő rendszer elkészítése egy választott, valós témakör adatainak nyilvántartására, majd ezeknek az adatoknak a feldolgozása Java nyelven, a DOM (Document Object Model) szabvány eszközeivel.

A munka két részből áll. Az első feladatban az adatbázist ER modellben terveztem meg, ezt XDM modellre konvertáltam, majd a modell alapján elkészítettem az XML dokumentumot és az azt ellenőrző XMLSchema-t. A második feladatban egy Java DOM program olvassa be, kérdezi le és módosítja az XML dokumentum adatait.

A választott téma egy gabonatelep (terménytároló és -forgalmazó telep) nyilvántartása. A témát a családi gazdálkodói háttér miatt választottam: a terményátvétel, a mérlegelés és a minőségvizsgálat folyamatát a gyakorlatból ismerem.

A feladat leírása

Egy gabonatelep a környékbeli gazdáktól (termelőktől) veszi át a learatott terményt, például búzát és kukoricát, tárolja a raktáraiban (silókban, tárolókban), majd a vevőknek, például malmoknak és takarmánygyártóknak szállítja tovább. A rendszer célja, hogy a telep minden be- és kiszállítást, a raktárkészlet változását és az átvett termény minőségét visszakereshetően nyilvántartsa.

A folyamat központi eleme a mérlegelés. Minden jármű a telepre érkezéskor és távozáskor áthalad a hídmérlegen. A rakott jármű tömege a bruttó tömeg, az üres járműé az üres (tára) tömeg, a kettő különbsége a nettó tömeg, vagyis a ténylegesen szállított termény mennyisége. Minden mérlegelésről mérlegjegy készül, amely rögzíti a jármű rendszámát, a tömegadatokat mázsában, a szállítás irányát (beszállítás vagy kiszállítás) és dátumát, valamint hogy melyik gazdához, melyik vevőhöz és melyik terményhez tartozik.

A gazdákról a telep az adószámot, a nevet, a lakcímet és az elérhetőséget tartja nyilván. Az elérhetőség összetett adat: telefonszámból, opcionális faxszámból és egy vagy több e-mail-címből áll. A vevőknél a név és az elérhetőség mellett a szállítási cím is szerepel. A terményeknél a nevet és a fajtát rögzítjük (például búza, Mv Nádor fajta).

A raktáraknak neve, címe és mázsában megadott kapacitása van. A raktárkészlet változását a készletmozgások írják le: minden mozgás megadja, melyik raktárba vagy raktárból, melyik terményből, mekkora mennyiség mozgott, milyen irányban és mikor, illetve hogy melyik

mérlegjegy alapján történt. A készletmozgás így a raktárak és a termények közötti több-több (N:M) kapcsolatot valósítja meg, saját tulajdonságokkal.

A beszállított terményből a telep laborja mintát vesz. A minőségvizsgálat a nedvességtartalmat és az idegenanyag-tartalmat százalékban rögzíti, valamint a vizsgálat dátumát, a laboráns nevét és a megállapított minőségi osztályt (javító, malmi vagy takarmány). Egy mérlegjegyhez legfeljebb egy minőségvizsgálat tartozik. A túl nedves termény szárítás nélkül nem tárolható biztonságosan, ezért a nedvesség az egyik legfontosabb lekérdezési szempont.

A második feladatban a Java program a teljes dokumentumot beolvassa és blokkos formában kiírja, lekérdezéseket futtat rajta (például a beérkező szállítások, a gyenge minőségű tételek gazdái vagy a raktáronkénti készlet), végül a példányokon módosításokat hajt végre.

1. Az XML adatkezelő rendszer tervezése és megvalósítása

1.1 Az adatbázis ER modell tervezése

Az ER modellt az ERDPlus szerkesztőprogrammal készítettem, szabványos jelölésekkel (1. ábra). A modell hét egyedből áll: Merlegjegy, Gazda, Vevo, Termeny, Raktar, KeszletMozgas és MinosegVizsgalat. Az egyedeket téglalap, a tulajdonságokat ellipszis, a kapcsolatokat rombusz jelöli. Az elsődleges kulcsok aláhúzással szerepelnek (például `Merlegjegy_ID`, `Gazda_ID`).

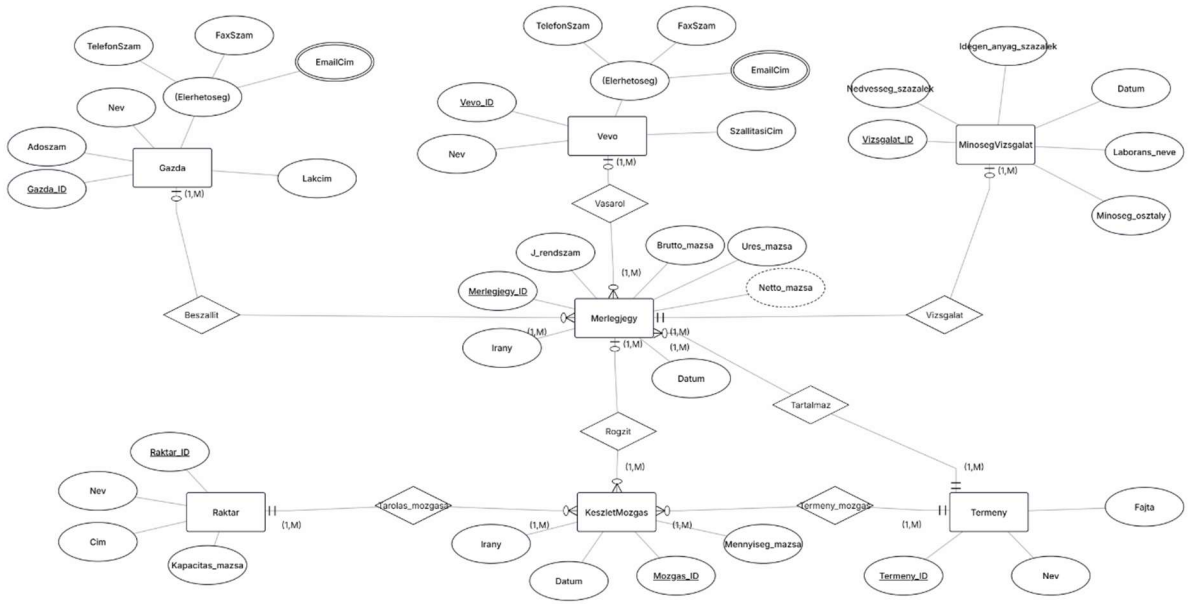
A tulajdonságok típusai:

- **Normál tulajdonság:** például `J_rendszam`, `Brutto_mazsa`, `Lakcim`.
- **Kulcs tulajdonság:** minden egyed saját azonosítója (`*_ID`), aláhúzva.
- **Összetett tulajdonság:** az `Elerhetoseg` a Gazda és a Vevo egyednél, amely a `TelefonSzam`, `FaxSzam` és `EmailCim` részekből áll.
- **Többértékű tulajdonság:** az `EmailCim` (dupla ellipszis), mert egy gazdának vagy vevőnek több e-mail-címe is lehet.
- **Származtatott tulajdonság:** a `Netto_mazsa` (szaggatott ellipszis), mert a bruttó és az üres tömeg különbségéből számolható.

A kapcsolatok:

- **1:N kapcsolatok:** egy gazda több mérlegjegyen szerepelhet (Beszallit), egy vevő is több mérlegjegyen (Vasarol). Egy terményről több mérlegjegy készülhet (Tartalmaz), és egy mérlegjegy alapján készletmozgás rögzíthető (Rogzit).

- **1:1 kapcsolat:** egy mérlegjegyhez legfeljebb egy minőségvizsgálat tartozik (Vizsgalat).
- **N:M kapcsolat:** egy raktárban több féle termény tárolható, és egy termény több raktárban is lehet. Ezt a KeszletMozgas kapcsolóegyed oldja fel (Tarolas_mozgasa és Termeny_mozgas), amelynek saját tulajdonságai is vannak: `Irany`, `Datum`, `Mennyiseg_mazsa`.



1. ábra: A gabonatelep ER modellje

Az ER modell: ILQ2OX_ER.jpg (GitHub)

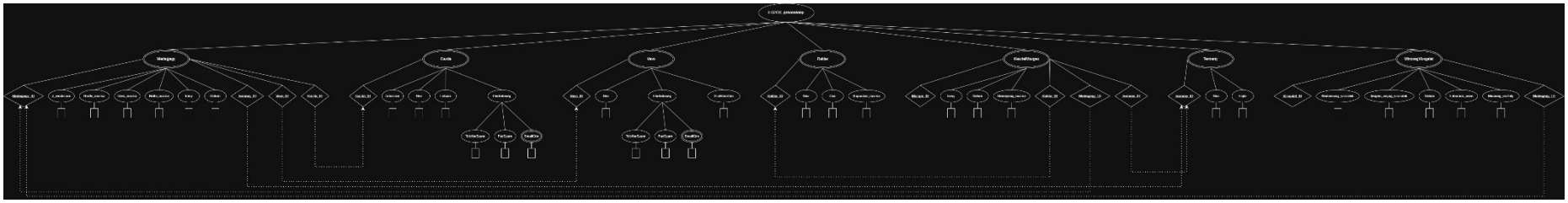
1.2 Az adatbázis konvertálása XDM modellre

Az XDM modellt a draw.io szerkesztőben készítettem el az ER modell alapján (2. ábra). A konverzió a következő szabályokat követi:

- A modell gyökéreleme az `ILQ2OX_gabonatelep`, minden egyed ennek a közvetlen gyereke.
- Az egyedekből elemek lesznek. Mivel mindegyikből több példány lehet, dupla ellipszis jelöli őket (többszörös előfordulás).
- A kulcsok attribútumok lesznek (rombusz). Az elsődleges kulcs aláhúzott, az idegen kulcs szaggatott nyíllal mutat a hivatkozott elem elsődleges kulcsára.
- A normál tulajdonságok szöveges tartalmú gyerekelemek lesznek, a szöveges tartalmat téglalap jelöli.
- Az összetett `Elerhetoseg` tulajdonság beágyazott elem lett, a részei az ő gyerekelemei.
- A többértékű `EmailCim` többször előforduló elemként szerepel (dupla ellipszis).

- Az 1:N kapcsolatokat az N oldali elem idegen kulcsa valósítja meg. Ezért a Merlegjegy elemnek `Termeny_ID`, `Vevo_ID` és `Gazda_ID` attribútuma van.
- Az 1:1 kapcsolatban a MinosegVizsgalat kapta a `Merlegjegy_ID` idegen kulcsot. Az egyediséget az XMLSchema `unique` megszorítása biztosítja.
- Az N:M kapcsolat önálló elem lett (KeszletMozgas). Két idegen kulcsa (`Raktar_ID`, `Termeny_ID`) mellett a kapcsolat saját tulajdonságait is tartalmazza, a `Merlegjegy_ID` pedig a mérlegjegyre hivatkozik.

A modell nagyon széles, ezért a 2. ábra külön, fekvő tájolású oldalra került.



2. ábra: A gabonatelep XDM modellje

Az XDM modell: ILQ2OX_XDM.drawio (GitHub)

1.3 Az XDM modell alapján XML dokumentum készítése

Az XML dokumentum (`ILQ2OX_XML.xml`) az XDM modell szerkezetét követi. A gyökérelem az `ILQ2OX_gabonatelep`, a gyerekei egyedtípusonként csoportosítva következnek, és minden csoport előtt megjegyzés áll. A Merlegjegy elemből három, a többi többször előforduló elemből két-két példányt készítettem. A többértékű `EmailCim` elemből is van olyan példány, amelyben két érték szerepel. A gyökérelem `xsi:noNamespaceSchemaLocation` attribútuma köti a dokumentumot a sémához.

A kulcsok és az idegen kulcsok attribútumként szerepelnek, a tulajdonságok gyerekelemként. Az alábbi részlet egy mérlegjegyet és egy két e-mail-címmel rendelkező gazdát mutat:

```
<!-- Gyökérelem -->
<ILQ2OX_gabonatelep xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
                    xsi:noNamespaceSchemaLocation="ILQ2OX_XMLSchema.xsd">

    <!-- Mérlegjegyek -->
    <Merlegjegy Merlegjegy_ID="mj_01" Termeny_ID="t_01" Vevo_ID="v_01" Gazda_ID="g_01">
        <J_rendszam>ABC-123</J_rendszam>
        <Brutto_mazsa>420</Brutto_mazsa>
        <Ures_mazsa>150</Ures_mazsa>
        <Netto_mazsa>270</Netto_mazsa>
        <Irany>Be</Irany>
        <Datum>2026-07-12</Datum>
    </Merlegjegy>
    <!-- Gazdák -->
    <Gazda Gazda_ID="g_02">
        <Adoszam>87654321-1-04</Adoszam>
        <Nev>Szabó Péter</Nev>
        <Lakcim>5700 Gyula, Kossuth tér 3.</Lakcim>
        <Elerhetoseg>
            <TelefonSzam>+36 20 444 5566</TelefonSzam>
            <FaxSzam>+36 66 654 321</FaxSzam>
            <EmailCim>szabo.peter@gmail.com</EmailCim>
            <EmailCim>peter.szabo@agrar.hu</EmailCim>
        </Elerhetoseg>
    </Gazda>
```

Az N:M kapcsolatot megvalósító KeszletMozgas elem négy kulcsattribútummal rendelkezik: a saját elsődleges kulcsán kívül három idegen kulcsa van.

```
<!-- Készletmozgások -->
<KeszletMozgas Mozgas_ID="km_01" Raktar_ID="r_01" Merlegjegy_ID="mj_01" Termeny_ID="t_01">
    <Irany>Be</Irany>
    <Datum>2026-07-12</Datum>
    <Mennyiseg_mazsa>270</Mennyiseg_mazsa>
</KeszletMozgas>
```

Az XML dokumentum: ILQ2OX_XML.xml (GitHub)

1.4 Az XML dokumentum alapján XMLSchema készítése

Az XMLSchema (`ILQ2OX_XMLSchema.xsd`) a következő lépésekben készült el:

- **Saját egyszerű típusok:** a tulajdonságok értékkészletét korlátozzák (minta, felsorolás, értéktartomány).
- **Globális elemek:** a több egyedben is előforduló elemek (`Nev`, `Datum`, `Irany`, `Elerhetoseg`) egyszer vannak definiálva, és `ref` hivatkozással kerülnek be a komplex típusokba.
- **Komplex típusok:** minden egyed saját komplex típust kapott. A gyerekelemek sorrendjét `xs:sequence` rögzíti; a Termeny típusnál `xs:all` szerepel, így ott a sorrend tetszőleges.
- **Kulcsok:** a gyökérelemben `xs:key` definiálja az elsődleges kulcsokat, `xs:keyref` az idegen kulcsokat, az 1:1 kapcsolatot pedig `xs:unique` biztosítja.

A saját típusok közül néhány: a rendszám formátumát reguláris kifejezés ellenőrzi, a tömeg nemnegatív egész szám, az irány és a minőségi osztály pedig felsorolás.

```
<!-- Egyszerű típusok -->
<xs:simpleType name="rendszamTipus">
    <xs:restriction base="xs:string">
        <xs:pattern value="[A-Z]{3}-\d{3}"/>
    </xs:restriction>
</xs:simpleType>

<xs:simpleType name="mazsaTipus">
    <xs:restriction base="xs:nonNegativeInteger">
        <xs:maxInclusive value="100000"/>
    </xs:restriction>
</xs:simpleType>

<xs:simpleType name="iranyTipus">
    <xs:restriction base="xs:string">
        <xs:enumeration value="Be"/>
        <xs:enumeration value="Ki"/>
    </xs:restriction>
</xs:simpleType>
```

Az összetett Elerhetoseg típusban a `FaxSzam` elhagyható (`minOccurs="0"`), az `EmailCim` pedig tetszőleges számban ismétlődhet (`maxOccurs="unbounded"`). Ez a többértékű tulajdonság sémabeli megfelelője.

```
<!-- Összetett tulajdonság, EmailCim többértékű -->
<xs:complexType name="elerhetosegTipus">
    <xs:sequence>
        <xs:element name="TelefonSzam" type="telefonTipus"/>
        <xs:element name="FaxSzam" type="telefonTipus" minOccurs="0"/>
        <xs:element name="EmailCim" type="emailTipus" maxOccurs="unbounded"/>
    </xs:sequence>
</xs:complexType>
```

A kulcsokat a gyökérelem definíciójában adtam meg. A `selector` kiválasztja az elemeket, a `field` megadja, melyik attribútum a kulcs. A `keyref` azt ellenőrzi, hogy az idegen kulcs értéke

létező elsődleges kulcsra mutasson, a `unique` pedig azt, hogy egy mérlegjegyhez legfeljebb egy minőségvizsgálat tartozzon:

```
<!-- PK Kulcsok -->
<xs:key name="merlegjegy_kulcs">
    <xs:selector xpath="Merlegjegy"/>
    <xs:field xpath="@Merlegjegy_ID"/>
</xs:key>

<!-- Idegen kulcsok -->
<xs:keyref refer="gazda_kulcs" name="merlegjegy_gazda_idegen_kulcs">
    <xs:selector xpath="Merlegjegy"/>
    <xs:field xpath="@Gazda_ID"/>
</xs:keyref>

<!-- 1-1 kapcsolat -->
<xs:unique name="unique_vizsgalat">
    <xs:selector xpath="MinosegVizsgalat"/>
    <xs:field xpath="@Merlegjegy_ID"/>
</xs:unique>
```

Az XML dokumentum a séma alapján érvényes (validált).

Az XMLSchema: ILQ2OX_XMLSchema.xsd (GitHub)

2. A DOM program készítése

A második feladat egy Java nyelvű DOM program, amely az `ILQ2OX_XML.xml` dokumentum adatait olvassa, kérdezi le és módosítja. A program a JDK beépített `javax.xml.parsers` és `org.w3c.dom` csomagjait használja, külső könyvtár nélkül. A fejlesztés Java 21-gyel, VS Code-ban történt.

- **Project name:** ILQ2OX_DOMParse
- **Package:** ilq2ox.domparse.hu
- **Class names:** ILQ2OX_DOMRead, ILQ2OX_DOMQuery, ILQ2OX_DOMModify
- **XML dokumentum:** ILQ2OX_XML.xml

A három osztály közös segédmetódusokat használ: a `getText()` és a `removeWhitespaceNodes()` a DOMRead osztályban, a `keresID()` a DOMQuery osztályban van. A többi osztály ezeket statikus importtal éri el, így egy-egy művelet csak egy helyen van megírva.

2.1 Adatolvasás

Az `ILQ2OX_DOMRead` osztály a teljes dokumentumot beolvassa, minden elemet blokk formában kiír a konzolra, végül a DOM-fát új fájlba (`ILQ2OX_XML_out.xml`) menti.

A beolvasás három lépésből áll. A `DocumentBuilderFactory` gyár hozza létre a `DocumentBuilder` feldolgozót, ez elemzi a fájlt, az eredmény pedig a memóriában felépített teljes DOM-fa (`Document`):

```
// A gyár (factory) hozza létre a feldolgozót, a beállításokat ezen lehet megadni
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();

// A DocumentBuilder végzi a tényleges elemzést
DocumentBuilder builder = factory.newDocumentBuilder();

// A fájl beolvasása és elemzése: az eredmény a teljes DOM-fa a memóriában
Document doc = builder.parse(xmlFile);

// Normalizálás: az egymás melletti szöveges csomópontokat összevonja
doc.getDocumentElement().normalize();
```

Ezután minden egyedtípust egy ciklus dolgoz fel. A `getElementsByTagName()` a dokumentum összes adott nevű elemét adja vissza egy `NodeList`-ben. A kulcsokat a `getAttribute()` olvassa ki, a gyerekelemek szövegét a `getText()` segédmetódus:

```
NodeList merlegjegyList = doc.getElementsByTagName("Merlegjegy");

for (int i = 0; i < merlegjegyList.getLength(); i++) {
    Node node = merlegjegyList.item(i);
    if (node.getNodeType() == Node.ELEMENT_NODE) {

        // A Node-ot Element-té alakítjuk, hogy elérjük az attribútumait
        Element elem = (Element) node;

        // Az elsődleges és az idegen kulcsok attribútumként szerepelnek
        String merlegjegyId = elem.getAttribute("Merlegjegy_ID");
        // A gyerekelemek szöveges tartalma
        String rendszam = getText(elem, "J_rendszam");
        System.out.println("Mérlegjegy azonosító: " + merlegjegyId);
        System.out.println("Jármű rendszáma: " + rendszam);
    }
}

// Visszaadja az elem adott nevű első gyerekelemének szöveges tartalmát, a széleiről levágott
// szóközökkel (az XSD whiteSpace=collapse miatt pl. " 200 " is érvényes érték lehet).
// Ha nincs ilyen gyerekelem, üres szöveget ad vissza (így nem kapunk NullPointerException-t).
public static String getText(Element elem, String tagName) {
    Node node = elem.getElementsByTagName(tagName).item(0);
    return node != null ? node.getTextContent().trim() : "";
}
```

A többértékű `EmailCim` elemeket a `printElerhetoseg()` metódus egy belső ciklussal fűzi össze egy sorba. Egy blokk kimenete a konzolon:

```
Aktuális elem: Gazda
Gazda azonosító: g_02
Adószám: 87654321-1-04
Név: Szabó Péter
Lakcím: 5700 Gyula, Kossuth tér 3.
Telefonszám: +36 20 444 5566
Faxszám: +36 66 654 321
E-mail cím(ek): szabo.peter@gmail.com, peter.szabo@agrar.hu
```

A mentést a `Transformer` végzi: a forrás (`DOMSource`) a DOM-fa, az eredmény (`StreamResult`) a fájlba író stream. Mentés előtt a `removeWhitespaceNodes()` törli az eredeti behúzásból maradt

üres szöveges csomópontokat, így a Transformer egységesen, 4 szóközzel húzhatja be a kimenetet:

```
// A Transformer a DOM-fát (forrás) szöveges XML-lé alakítja (eredmény)
TransformerFactory transformerFactory = TransformerFactory.newInstance();
Transformer transformer = transformerFactory.newTransformer();

// Kimeneti beállítás: behúzás 4 szóközzel.
transformer.setOutputProperty(OutputKeys.INDENT, "yes");
transformer.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");

// Az XML-deklarációt kézzel írjuk ki, mert a Transformer nem tesz utána sortörést
transformer.setOutputProperty(OutputKeys.OMIT_XML_DECLARATION, "yes");

// A try-with-resources a végén automatikusan lezárja a fájlt
try (Writer writer = new OutputStreamWriter(new FileOutputStream(fileName),
StandardCharsets.UTF_8)) {
    writer.write("<?xml version=\"1.0\" encoding=\"UTF-8\"?>" + System.lineSeparator());

    // Forrás (DOMSource): a DOM-csomópont; eredmény (StreamResult): a fájlba író writer.
    NodeList topLevel = doc.getChildNodes();
    for (int i = 0; i < topLevel.getLength(); i++) {
        transformer.transform(new DOMSource(topLevel.item(i)), new StreamResult(writer));
    }
}
```

Forráskód: ILQ2OX_DOMRead.java (GitHub)

2.2 Adat-lekérdezés

Az `ILQ2OX_DOMQuery` osztály négy lekérdezést futtat a dokumentum példányain, és az eredményt a konzolra írja. A feladatkiírásnak megfelelően a lekérdezések nem használnak XPath-kifejezést: kizárólag DOM-bejárással dolgoznak (`getElementsByTagName()`, `getAttribute()`, `getTextContent()`).

- **a) Beérkező szállítások:** azok a mérlegjegyek, amelyeknél az `Irany` értéke `Be`. Mindegyiknél kiírja a rendszámot, a dátumot és a nettó tömeget.
- **b) Gazdák és vevők e-mail-címei:** a többértékű `EmailCim` elem bejárása. Ugyanaz a metódus dolgozza fel a Gazda és a Vevo elemeket, a tagnevet paraméterként kapja.
- **c) Beérkező, gyenge minőségű tételek gazdája:** azok a minőségvizsgálatok, amelyeknél a nedvesség 14% fölött van. A lekérdezés két lépésben kapcsolja össze az adatokat: a vizsgálat `Merlegjegy_ID` idegen kulcsával megkeresi a mérlegjegyet, majd annak `Gazda_ID` idegen kulcsával a gazdát.
- **d) Raktáronkénti aktuális készlet:** egymásba ágyazott ciklus. A külső ciklus a raktárakon, a belső a készletmozgásokon halad végig. A beérkező mennyiséget hozzáadja, a kiszállítottat levonja, végül a kapacitáshoz viszonyított telítettséget is kiszámolja.

Az összekapcsoláshoz a `keresID()` segédmetódus kell, amely elemnév, attribútumnév és érték alapján keres meg egy elemet. Ezt a DOMModify osztály is használja:

```
// Keresés az XML-ben egy elemre, amelynek adott attribútuma adott értékű.
public static Element keresID(Document doc, String tagName, String attrName, String ertek) {
    NodeList nList = doc.getElementsByTagName(tagName);

    for (int i = 0; i < nList.getLength(); i++) {
        Node nNode = nList.item(i);
        if (nNode.getNodeType() == Node.ELEMENT_NODE) {
            Element elem = (Element) nNode;
            if (elem.getAttribute(attrName).equals(ertek)) {
                return elem;
            }
        }
    }
    return null; // Ha nincs találat
}
```

A c) lekérdezés lényege az idegen kulcsokon keresztüli kétlépéses keresés:

```
double nedvesseg = Double.parseDouble(getText(elem, "Nedvesseg_szazalek"));
// Megnézzük, hogy a nedvesség meghaladja-e a határértéket
if (nedvesseg > NEDVESSEG_HATAR) {
    // Ha igen, akkor a mérlegjegy azonosítóját változóba mentjük
    String merlegjegyID = elem.getAttribute("Merlegjegy_ID");
    // 1. lépés: a vizsgálat idegen kulcsával megkeressük a mérlegjegyet
    Element merlegjegyElem = keresID(doc, "Merlegjegy", "Merlegjegy_ID", merlegjegyID);
    // Csak a beérkező (Be) szállítás gazdája felelős a minőségért;
    // kiszállításnál (Ki) a tétel már a telepről megy a vevőhöz
    if (!getText(merlegjegyElem, "Irany").equals("Be")) {
        continue;
    }
    // 2. lépés: a mérlegjegy Gazda_ID-jével megkeressük a gazdát
    String gazdaID = merlegjegyElem.getAttribute("Gazda_ID");
    Element gazdaElem = keresID(doc, "Gazda", "Gazda_ID", gazdaID);
}
```

A d) lekérdezés belső ciklusa összegzi az adott raktárhoz tartozó mozgásokat:

```
// Belső ciklus: az összes mozgás közül az ehhez a raktárhoz tartozókat összegezzük
for (int j = 0; j < mozgasList.getLength(); j++) {
    Node mozgasNode = mozgasList.item(j);
    if (mozgasNode.getNodeType() == Node.ELEMENT_NODE) {
        Element mozgasElem = (Element) mozgasNode;
        String mozgasRaktarID = mozgasElem.getAttribute("Raktar_ID");
        if (mozgasRaktarID.equals(raktarID)) {
            int mennyiseg = Integer.parseInt(getText(mozgasElem, "Mennyiseg_mazsa"));
            String irany = getText(mozgasElem, "Irany");
            if (irany.equals("Be")) {
                aktualisKeszlet += mennyiseg;
            } else if (irany.equals("Ki")) {
                aktualisKeszlet -= mennyiseg;
            } else {
                // Ismeretlen irány: nem számoljuk bele, de jelezzük
            }
        }
    }
}
```

A c) és d) lekérdezés kimenete:

```
c.) Beérkező gyenge minőségű tételek gazdája:
Gyenge minőségű tétel: Merlegjegy_ID=mj_02, Rendszám: DEF-456, Gazda neve: Szabó Péter,
Nedvesség: 15.8%

d.) Raktáronkénti aktuális készlet:
Raktár: Központi siló, Aktuális készlet: 70 / 5000, Telítettség: 1.40%
Raktár: Északi tároló, Aktuális készlet: 245 / 2500, Telítettség: 9.80%
```

Forráskód: ILQ2OX_DOMQuery.java (GitHub)

2.3 Adatmódosítás

Az `ILQ2OX_DOMModify` osztály négy módosítást hajt végre a memóriában lévő DOM-fán, majd a módosított dokumentumot a konzolra írja. Az eredeti XML fájl változatlan marad. A módosítások úgy készültek, hogy a dokumentum utánuk is érvényes maradjon a séma szerint.

- **a) Elem szöveges tartalmának módosítása:** az `mj_02` mérlegjegy újramérése. Az új üres tömegből a program újraszámolja a nettó tömeget, és a mérlegjegyhez tartozó készletmozgás mennyiségét is frissíti, hogy az adatok ne mondjanak ellent egymásnak (`setTextContent()`).
- **b) Új elem beszúrása:** új e-mail-cím felvétele a `g_01` gazdához (`createElement()`, `appendChild()`).
- **c) Attribútum (idegen kulcs) módosítása:** az `mj_03` kiszállítás vevőjének cseréje. Csere előtt a program ellenőrzi, hogy az új vevő létezik-e, mert az idegen kulcs csak létező elsődleges kulcsra mutathat (`setAttribute()`).
- **d) Elem törlése:** a vevők faxszámának törlése (`removeChild()`). A `FaxSzam` a sémában opcionális, így a törlés után is érvényes a dokumentum.

Új elem beszúrásakor az elemet a `Document` hozza létre, de ekkor még nincs benne a fában. Az `appendChild()` fűzi be a szülő utolsó gyerekeként:

```
// Az Elerhetoseg összetett elem, az e-mail címek ennek a gyerekei
Element elerhetoseg = (Element) gazda.getElementsByTagName("Elerhetoseg").item(0);

// Az új elemet a Document hozza létre; ekkor még nincs benne a fában
Element ujElem = doc.createElement("EmailCim");
ujElem.setTextContent(email);

// Az appendChild() az utolsó gyerek után fűzi be az elemet. Az XSD-ben az EmailCim
// a sorozat utolsó eleme, így a sorrend a beszúrás után is érvényes marad.
elerhetoseg.appendChild(ujElem);
```

Az idegen kulcs módosítása előtt a program ellenőrzi a hivatkozott elem létezését:

```
// Az idegen kulcs csak létező vevőre mutathat (az XSD keyref ezt ellenőrzi),
// ezért módosítás előtt megnézzük, hogy van-e ilyen vevő
Element ujVevo = keresID(doc, "Vevo", "Vevo_ID", ujVevoId);
if (ujVevo == null) {
    System.out.println("Hiba: nem található a vevő: " + ujVevoId);
    return;
}

String regiVevoId = merlegjegy.getAttribute("Vevo_ID");

// A setAttribute() a meglévő attribútum értékét felülírja (ha nem létezne, létrehozná)
merlegjegy.setAttribute("Vevo_ID", ujVevoId);
```

Törléskor arra kell figyelni, hogy a `getElementsByTagName()` élő listát ad vissza, amely a törléssel rövidül. Ezért a ciklus hátulról halad, és a törlést a szülő végzi:

```
// A NodeList élő lista: törléskor rövidül, ezért hátulról haladunk
NodeList faxList = vevo.getElementsByTagName("FaxSzam");
for (int j = faxList.getLength() - 1; j >= 0; j--) {
    Node fax = faxList.item(j);

    // Egy elemet csak a szülője törölhet, ezért a getParentNode()-on hívjuk a removeChild()-ot
    fax.getParentNode().removeChild(fax);
}
```

A módosítások kimenete a konzolon (utána a program a teljes módosított dokumentumot is kiírja):

```
a.) Az mj_02 mérlegjegy újramérése (új üres tömeg: 150 mázsa):
Mérlegjegy: mj_02, Bruttó: 385 mázsa
Üres tömeg: 140 -> 150 mázsa
Nettó tömeg: 245 -> 235 mázsa
Készletmozgás: km_02, Mennyiség: 245 -> 235 mázsa

b.) Új e-mail cím felvétele a g_01 gazdához:
Gazda: Kovács János (g_01)
Új e-mail cím: kovacs.janos@agrar.hu
E-mail címek száma: 2

c.) Az mj_03 kiszállítás vevőjének cseréje v_01-re:
Mérlegjegy: mj_03, Rendszám: GHI-789
Vevo_ID: v_02 -> v_01 (Alföldi Malom Zrt.)

d.) A vevők faxszámának törlése:
Vevő: Alföldi Malom Zrt., törölt faxszám: +36 62 555 101
Vevő: Dél-Tiszai Takarmány Kft., törölt faxszám: +36 66 777 201
```

Forráskód: ILQ2OX_DOMModify.java (GitHub)